



SIGNAL SERIES • LABS

LABS HANDBOOK

Build it, break it, recover it — hands on.

By *Nabawy*

AUTHOR & COMPILER

Lab 01: First Pipeline

Branch Protection

Lab 02: Build And Test

Hermetic Builds

System Map

Table of Contents

04	Lab 01: First Pipeline	04	05	Branch Protection	05
06	Lab 02: Build And Test	06	08	Hermetic Builds	08
09	Hermetic Builds · Map	09	10	Lab 03: Artifacts	10
12	Sbom & Verification	12	13	Lab 04: Deployment	13
14	The Production Gate	14	15	The Production Gate · Map	15
16	Lab 05: Rollback	16	17	Rollback & Postmortem	17
18	Lab 06: Jenkins Architecture	18	19	Canonical Pipeline	19
20	Canonical Pipeline · Map	20	21	Lab 07: Shared Library	21
22	Library Consumption	22	23	Library Consumption · Map	23
24	Lab 08: Backup & Restore	24	25	Backup & Restore	25
26	Backup & Restore · Map	26	27	Automated Triggering	27
28	Configuration Pre-Flight	28	29	Network Reachability	29
30	Pipeline Construction	30	31	Lab 01: Authenticated Checkout	31
32	The Webhook Bridge	32	33	Lab 02: End-To-End Trigger	33
34	Lab 02 End-To-End Trigger · Map	34	35	Container Delivery	35
36	Build Architecture	36	37	Secure Authentication	37
38	Pipeline Definition	38	39	Simulated Deployment	39
40	Lab: End-To-End Delivery	40	41	End-To-End Delivery · Map	41
42	Multibranch Pipelines	42	43	Scan Architecture	43
44	Job Configuration	44	45	Pipeline Logic	45
46	Lab: Lifecycle Management	46	47	Lifecycle Management · Map	47
48	Shared Libraries	48	49	Library Structure	49
50	Global Configuration	50	51	Scripted Execution	51
52	Declarative Execution	52	53	Lab: Conditional Validation	53
54	Conditional Validation · Map	54	55	Infrastructure Pipeline	55
56	Provisioning Architecture	56	57	Environment Isolation	57
58	Manual Approval Gate	58	59	Simulated Cloud	59
60	Lab: Provisioning & Recovery	60	61	Provisioning & Recovery · Map	61
62	 Quick Revision — Terms Table	62	63	 Concept Questions — Distilled Answers	63
64	 Command Cheat Sheet	64	65	 Self-Quiz + Answer Key	65

▪ How to use this book legend

 NOTE · context  KEY · must retain  WATCH OUT · gotcha  VERIFIED · check

NOTE Read cover to cover once, then use the Contents to jump. Every chapter ends with a  EXAM box — if you can answer it cold, move on.

Threads: [Artifact Digest](#) · [Approval Gates](#) · [Rollback & Recovery](#)

Table of Contents · continued

34	Container Delivery	34	35	Build Architecture	35
36	Secure Authentication	36	37	Pipeline Definition	37
38	Simulated Deployment	38	39	Lab: End-To-End Delivery	39
40	System Map	40	41	Multibranch Pipelines	41
42	Scan Architecture	42	43	Job Configuration	43
44	Pipeline Logic	44	45	Lab: Lifecycle Management	45
46	System Map	46	47	Shared Libraries	47
48	Library Structure	48	49	Global Configuration	49
50	Scripted Execution	50	51	Declarative Execution	51
52	Lab: Conditional Validation	52	53	System Map	53
54	Infrastructure Pipeline	54	55	Provisioning Architecture	55
56	Environment Isolation	56	57	Manual Approval Gate	57
58	Simulated Cloud	58	59	Lab: Provisioning & Recovery	59
60	System Map	60	61	 Quick Revision — Terms Table	61
62	 Concept Questions — Distilled Answers	62	63	 Command Cheat Sheet	63
64	 Self-Quiz + Answer Key	64			

Threads: [Approval Gates](#)

Lab Map

Nine labs, one arc: first green check to Jenkins live to recovery drill. Read proves before needs — each row is the exam contract.

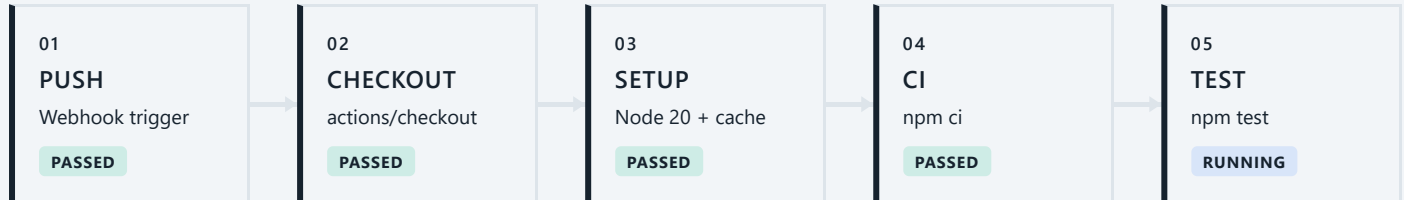
Lab	Proves	Needs
01 First Pipeline	Push turns green behind a gate	GitHub repo plus branch protection
02 Build and Test	DAG, cache, hermetic proof	Lab 01 green
03 Artifacts	SHA tags, registry, SBOM	Lab 02 hermetic build
04 Deployment	Staging auto, prod approval	Lab 03 digest
05 Rollback	Digest redeploy plus postmortem	Lab 04 release
06 Jenkins Controller	Compose, agent, first pipeline	Docker, Lab 01 habits
07 Shared Library	Versioned pipeline reuse	Lab 06 controller
08 Backup & Restore	DR drill restores service	Lab 06 plus Lab 07
Auth/Trigger extras	Webhooks, multibranch, IaC evidence	Labs 01–08 on demand

Next: [Lab 01: First Pipeline](#)

Lab 01: First Pipeline

01 / First Pipeline

Ship a green CI pipeline on GitHub Actions. The objective is to close the loop: trigger a pipeline via a push, execute a build and test sequence on a hosted runner, and surface the status as a check on the Pull Request.



OBJECTIVE

Establish the smallest closed loop in CI/CD: a developer pushes code, an automated system validates it, and the result is fed back to the source control platform.

ARCHITECTURE

Push → GitHub webhook → hosted runner (`ubuntu-latest`) → checkout → setup-node → `npm ci` → `npm test` → green check on PR.

Step 1: Add the Workflow

Create `.github/workflows/ci.yml` in the repository. This declarative YAML defines the trigger, the execution environment, and the sequence of steps.

```
name: ci

on: [push, pull_request]

jobs:

  build:

    runs-on: ubuntu-latest

    steps:

      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4

    with: { node-version: 20, cache: npm }

      - run: npm ci

      - run: npm test
```

EXPECTED RESULT

Push to a branch and open a PR. The Actions tab shows the run executing. The PR view displays a green checkmark indicating the status check passed.

Threads: [GitOps Pull Model](#) · [Controllers & Agents](#)

WHEN TO USE LAB TOOLS use when Docker freezes the runtime for labs · use when Jenkins executes the pipeline live · use when Terraform provisions disposable envs · avoid when production without hardening · avoid when secrets leave the lab machine. **Tool vs Tool — The Whole Field**

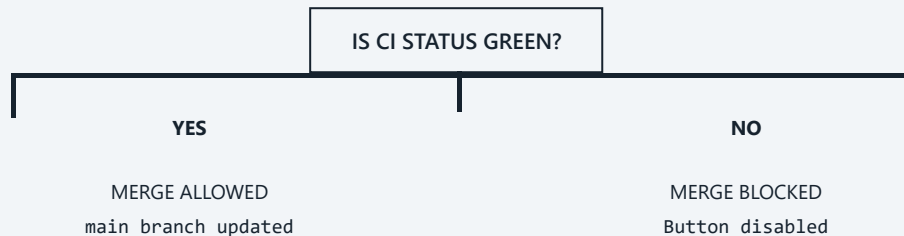
Branch Protection

02 / Enforce the Gate

An unenforced gate is decoration. A green check on a PR is meaningless if a developer can click "Merge" while the pipeline is red. Branch protection converts the CI result from a suggestion into a rule.

Step 2: Require the Check

Navigate to Repo Settings → Branches → protect `main`. Require the `build` status check to pass before merging.



Verification & Failure Scenarios

- ☐ PR shows green check from the workflow.
- ☐ Merging to `main` without green is blocked.
- ☐ Breaking the test turns the PR red.

GOTCHA

If `npm ci` fails, it usually means the lockfile is missing or out of sync. Run `npm install` locally, commit the `package-lock.json` file, and push again. Do not use `npm install` in CI; `npm ci` enforces hermetic, reproducible builds.

INTERVIEW CHECK

Q: What does the green check actually prove?

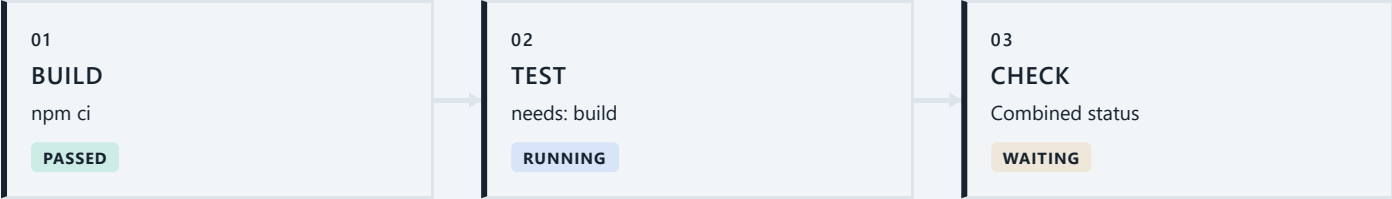
A: It proves that the specific code at that specific commit passed the specific checks defined in the YAML. It does not prove the application is bug-free. Breaking the test on purpose is the real verification—it proves the pipeline catches failures.

Threads: [Hermetic Builds](#)

Lab 02: Build And Test

03 / Build and Test DAG

Split Lab 01's single job into a fast, cached PR pipeline. By separating the build and test phases into distinct jobs, we create a Directed Acyclic Graph (DAG) that optimizes execution and isolates failures.



Lab 02: Build And Test • continued

OBJECTIVE

Create a hermetic build job followed by a unit test job. Total execution time should be under 10 minutes, with reruns benefiting from warm npm caches.

THE DAG EDGE

The `needs:` keyword in GitHub Actions defines the DAG edge. The `test` job will not execute until the `build` job completes successfully.

Step 1: Split and Cache

```
jobs:

  build:

    runs-on: ubuntu-latest

    steps:

      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4

    with: { node-version: 20, cache: npm }

      - run: npm ci

  test:

    needs: build

    runs-on: ubuntu-latest

    steps:

      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4

    with: { node-version: 20, cache: npm }

      - run: npm ci

      - run: npm test
```

Threads: [Hermetic Builds](#)

Hermetic Builds

04 / Hermeticity & Caching

A build is hermetic if it produces the exact same result regardless of the environment it runs in. In CI, this means the runner should not rely on ambient machine state. The lockfile is the contract.

Step 2: Prove Hermeticity

EXECUTE Delete `node_modules` locally. Run `npm ci && npm test`. The result should be identical to the CI runner.

VERIFY Two jobs visible in GitHub Actions, ordered by `needs`. Cache hit shown on rerun. Local `npm ci` reproduces CI result.

GOTCHA

If the cache never hits, the lockfile is likely not committed, or the key inputs have changed. The `actions/setup-node` action uses the lockfile to key the cache. If `package-lock.json` is in `.gitignore`, caching fails silently.

LOCAL COMMAND

```
$ rm -rf node_modules
$ npm ci
```

EXPECTED RESULT

```
added 42 packages, audited 43 packages
Identical to runner install
```

INTERVIEW CHECK

Q: What does `needs:` actually guarantee?

A: It guarantees that the downstream job will not start until the upstream job completes successfully. If the `build` job fails, the `test` job is skipped, saving compute time and preventing tests from running against a broken build.

Threads: [Hermetic Builds](#) · [Controllers & Agents](#)

Hermetic Builds • Map

05 / Pipeline System Map

The first two labs establish the foundational CI/CD loop. Understanding the components and their failure modes is critical before moving to artifacts and deployments.



FEEDBACK: PR STATUS CHECK & MERGE GATE ENFORCEMENT

Troubleshooting Flow

[ERROR] `npm ci` fails with 'lockfile missing'

-> Recovery: Commit package-lock.json

[ERROR] Runner queues indefinitely

-> Recovery: Free-tier concurrency limit; wait or reduce jobs

[ERROR] test runs before build finishes

-> Recovery: Add 'needs: build' to test job

[WARN] Cache never hits

-> Recovery: Ensure lockfile exists and cache key is correct

WHAT THIS PROVED

Triggers fire pipelines, runners execute jobs, checks gate merges, and lockfiles ensure hermetic, reproducible builds. The loop is closed. Continue to Lab 03 to package this build's output into a deployable artifact.

🎓 **EXAM** Built a green GitHub Actions loop on **ubuntu-latest** — **.github/workflows/ci.yml** with **actions/checkout@v4**, **actions/setup-node@v4**, **npm ci** then **npm test**; enforced it with branch protection on **main** so red can never merge; split it into a build-to-test DAG via **needs: build** with lockfile-keyed caching. Verified: the PR shows the green check, merging without green is blocked, breaking a test turns the PR red, and local **npm ci && npm test** reproduces the runner exactly — hermetic, no ambient state.

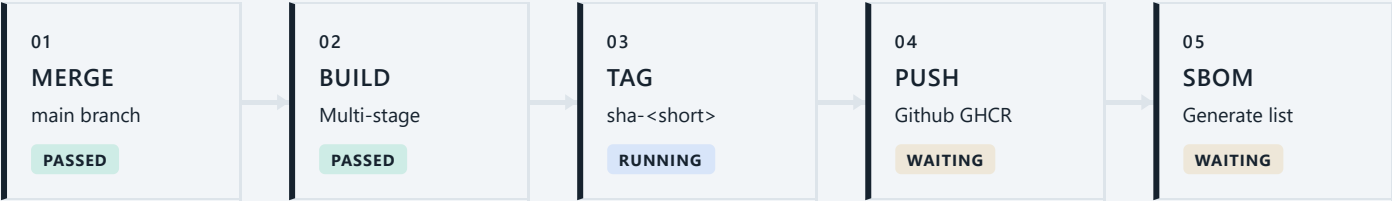
Next: Lab 03: Artifacts

Threads: [Hermetic Builds](#) • [Controllers & Agents](#)

Lab 03: Artifacts

01 / Artifact Immutability

Build a Docker image once, tag it by commit SHA, push it to GitHub Container Registry (GHCR), and attach a Software Bill of Materials (SBOM). The objective is to establish a single, immutable deployable identity that flows through all environments.



OBJECTIVE

Create an immutable artifact tagged by commit SHA, push it to a registry, and attach an SBOM. Prove that staging and production would run the exact same digest.

IMMUTABILITY

A tag like `:latest` is mutable; it can point to different bytes over time. A SHA tag (`sha-a1b2c3d`) and the underlying digest are permanent. The digest is the true deployable identity.

Threads: [Artifact Digest](#)

Lab 03: Artifacts · continued

Step 1: Build Once, Tag SHA

The workflow uses GitHub Actions to build the image, log into GHCR using the built-in `GITHUB_TOKEN`, tag the image with the commit SHA, and push it.

```
jobs:

  image:

    runs-on: ubuntu-latest

    permissions: { packages: write }

    steps:

      - uses: actions/checkout@v4

      - uses: docker/login-action@v3

    with: { registry: ghcr.io, username: ${ github.actor }, password: ${ secrets.GITHUB_TOKEN } }

      - uses: docker/metadata-action@v5

    id: meta

    with:

      images: ghcr.io/${ github.repository }

      tags: type=sha,prefix=sha-

      - uses: docker/build-push-action@v6

    with: { push: true, tags: ${ steps.meta.outputs.tags } }
```

EXPECTED RESULT

GHCR shows the `sha-<short>` tag. The GitHub Actions logs print the immutable image digest, which will be used for deployment.

Threads: [Artifact Digest](#)

Sbom & Verification

02 / Software Bill of Materials

An artifact is a black box. An SBOM (Software Bill of Materials) is the ingredient list. It makes the next CVE answerable in minutes, not days, by explicitly listing every package and dependency baked into the image.

Step 2: Attach SBOM

Add the `anchore/sbom-action` to the workflow immediately after the image is pushed. This action scans the image and attaches the resulting SBOM as a workflow artifact.

COMMAND

```
- uses: anchore/sbom-action@v0
with:
  image: ghcr.io/${{ github.repository }}:sha-${{
    github.sha }}
  format: spdx-json
```

RESULT

```
[Pipeline] stage
SBOM generated successfully.
Artifact: sbom-spdx.json
Packages scanned: 142
```

Verification & Failure Scenarios

- ☐ GHCR holds an immutable SHA tag.
- ☐ Same digest referenced twice equals same bytes.
- ☐ SBOM artifact present in the workflow run.

GOTCHA

If the push fails with a `403 Forbidden` error, the workflow is missing the `packages: write` permission. GitHub Actions tokens are least-privilege by default; you must explicitly grant write access to packages.

INTERVIEW CHECK

Q: Digest vs tag: why is redeploying `:latest` dangerous?

A: A tag is a pointer. If you deploy `:latest`, you are deploying whatever bytes the registry currently points to. If someone pushes a new image to `:latest`, your deployment silently changes. Pinning the digest (`sha256:...`) guarantees the exact bytes.

Threads: [Artifact Digest](#) · [Controllers & Agents](#)

Lab 04: Deployment

03 / Environment Promotion

Continuous Delivery means the software is always releasable, but release is a human decision. Promote Lab 03's immutable digest through staging to production with an explicit approval gate. Same digest, two environments.

STAGING	Auto-deploy on merge	ACTIVE
PRODUCTION	Manual approval required	GATED

Step 1: Staging Auto-Deploys

The staging deployment is fully automated. It depends on the `image` job from Lab 03 and deploys the specific digest output by that job.

```
deploy-staging:

  needs: image

  environment: staging

  runs-on: ubuntu-latest

  steps:

    - run: echo "deploy ${ needs.image.outputs.digest } to staging"
```

WHY IT MATTERS

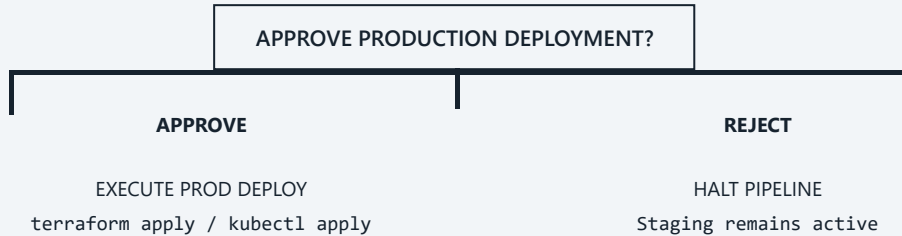
Staging proves the artifact continuously. Automation here is safe because users never see it. If the artifact fails in staging, the pipeline halts before reaching the production gate.

Threads: [Artifact Digest](#) · [Approval Gates](#)

The Production Gate

04 / The Production Gate

Production deployment requires a human decision. By utilizing GitHub Environments and required reviewers, the pipeline pauses automatically, waiting for approval before applying the same digest to production.



Step 2: Production Needs a Human

```
deploy-prod:

  needs: [deploy-staging]

  environment: production

  runs-on: ubuntu-latest

  steps:

    - run: echo "deploy ${ needs.image.outputs.digest } to prod"
```

- ☐ Staging deploys on every merge, no clicks.
- ☐ Production blocks until approved by a reviewer.
- ☐ Both environments ran the identical image digest.

GOTCHA

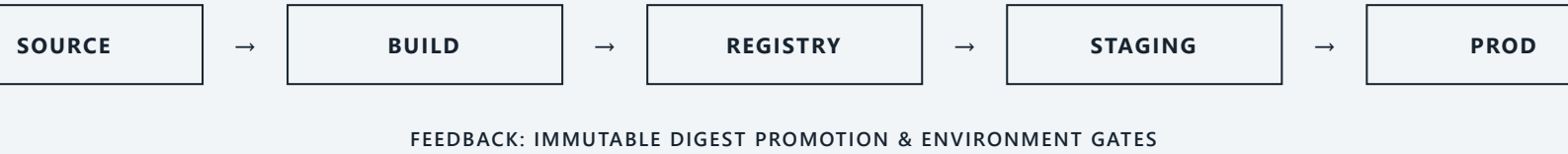
If digests differ between environments, the pipeline passed a tag (like `:latest` or `main`) instead of the explicit digest. Always pass `${ needs.image.outputs.digest }` explicitly to guarantee byte-for-byte parity.

Threads: [Approval Gates](#)

The Production Gate · Map

05 / System Map & Troubleshooting

The promotion path from source to production relies on strict immutability and environment isolation. Failures here usually stem from mutable tags or missing permissions.



Troubleshooting Flow

Symptom	Root Cause	Recovery
403 on push to GHCR	Missing permissions	Add <code>permissions: { packages: write }</code> to job
Prod deployed wrong code	Promoted a mutable tag	Pass <code>outputs.digest</code> explicitly
Approval never arrives	Missing environment config	Set required reviewers in GitHub Env settings
SBOM generation fails	Image inaccessible	Ensure SBOM action runs after image is pushed

INTERVIEW CHECK

Q: What does the production approval gate actually block?

A: It blocks the execution of the `deploy-prod` job. The artifact is already in the registry; the gate prevents the deployment command (e.g., `kubectl apply`) from running until a human clicks "Approve".

🔪 EXAM

Built one immutable artifact — image tagged `sha-<short>`, pushed to **GHCR** with **packages: write**, SBOM attached as **spx-json** via **anchore/sbom-action**; promoted the exact `${{ needs.image.outputs.digest }}` through auto-deploy **staging** to an approval-gated **production**. Verified: staging and production run the same digest — never **:latest** — and the SBOM answers the next CVE in minutes.

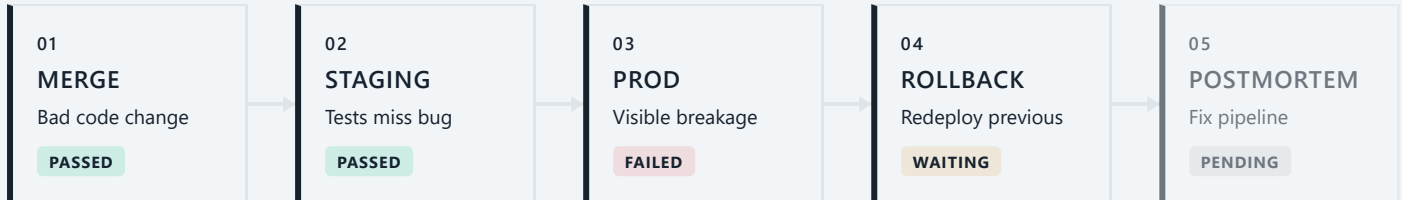
Next: Lab 05: Rollback

Threads: [Artifact Digest](#) · [Approval Gates](#)

Lab 05: Rollback

01 / The Incident & Recovery

Break production deliberately, then recover by redeploying the previous known-good digest. The objective is to prove that recovery is a practiced path, not an improvised fire drill. Green pipelines do not guarantee correct applications; tests only cover what you thought of.



OBJECTIVE

Cause an incident, recover by rolling back to a previously deployed artifact digest, and write a postmortem that results in a merged pipeline change.

WHY RECOVERY?

Recovery must be engineered. If your only recovery mechanism is "git revert and rebuild," you are adding untested variables to a fire. Old digests are proven; fresh builds are risks.

Step 1: Cause the Incident

Merge a change that passes all CI tests but breaks the application's visible output (e.g., changing a port or breaking a route). Promote this change to production.

EXPECTED RESULT

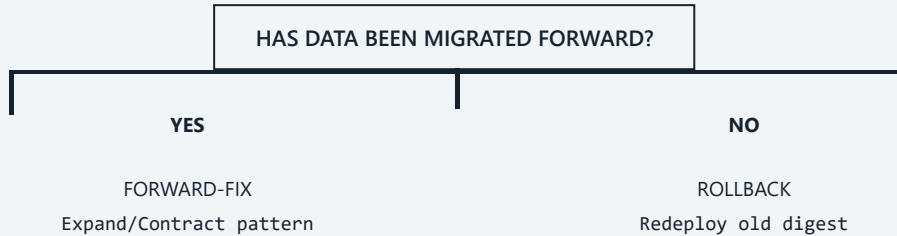
Production is broken despite a green pipeline. This proves the gap between "tests pass" and "application works."

Threads: [Rollback & Recovery](#) · [Artifact Digest](#)

Rollback & Postmortem

02 / Rollback & Postmortem

When production is broken, the fastest path to recovery is redeploying the last known-good state. Do not rebuild; redeploy the exact immutable digest that ran before the incident.



Step 2: Roll Back, Don't Rebuild

EXECUTE Redeploy the previous known-good digest to production. Re-run the old `deploy-prod` job or dispatch a new run with the saved digest value. Time yourself.

VERIFY Production is healthy again within minutes. No fresh build was involved.

Step 3: Postmortem with Teeth

```
Timeline: 10:00 Bad merge deployed

10:05 Users report 500 errors

10:07 Team declares incident

10:09 Rollback initiated (digest: sha-a1b2c3d)

10:12 Production healthy

Action Item: Add integration test for port binding
```

GOTCHA

If the previous digest is unknown, rollback is impossible. You must log every deployed digest in an immutable audit log. Without the digest, you are forced to rebuild, introducing untested variables into an active incident.

INTERVIEW CHECK

Q: Rollback vs forward-fix: which failure forces expand-contract instead of reverting?

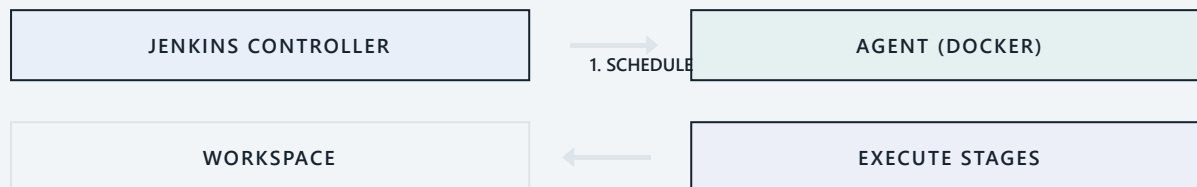
A: If a database migration has already executed and modified data schemas (forward-only migration), a code rollback will fail because the old code cannot read the new schema. You must forward-fix using an expand-contract pattern.

Threads: [Rollback & Recovery](#) · [Artifact Digest](#)

Lab 06: Jenkins Architecture

03 / Controller & Agent Split

Run a real Jenkins controller and agent using Docker Compose. The controller schedules work; the agent executes it. This split is the fundamental architecture of Jenkins, ensuring the brain of the system is never burdened by the muscle.



OBJECTIVE

Connect an inbound agent to a Jenkins controller and execute the canonical three-stage pipeline (Build, Test, Deploy) on the agent.

ZERO EXECUTORS

The controller must have zero executors. All builds must run on agents. Running builds on the controller risks crashing the entire CI system.

Docker Compose Setup

```
$ mkdir jenkins-lab && cd jenkins-lab

$ docker compose up -d

Creating jenkins ... done

Creating agent-1 ... done

$ docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword

e7f8a9c1d2e3f4g5h6i7j8k9l0m1n2o3
```

EXPECTED RESULT

Manage Jenkins → Nodes shows `agent-1` online with 2–4 idle executors. The controller (`master`) has zero executors.

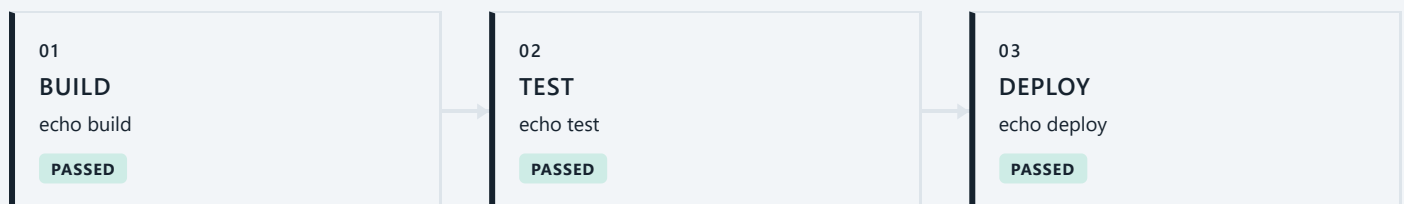
Threads: [Controllers & Agents](#)

Canonical Pipeline

04 / The Canonical Pipeline

With the agent connected, create a new Pipeline job. The pipeline must explicitly target the `docker-agent` label to prove scheduling works. The script contains the minimal three stages required to demonstrate execution and visualization.

```
pipeline {  
  
  agent { label 'docker-agent' }  
  
  stages {  
  
    stage('Build') { steps { sh 'echo build' } }  
  
    stage('Test') { steps { sh 'echo test' } }  
  
    stage('Deploy'){ steps { echo 'deploy' } }  
  
  }  
  
}
```



- ☐ Agent online, labeled `docker-agent` , executors idle after run.
- ☐ Three green stages visible in Stage View.
- ☐ Controller ran zero builds (check: no executors on `master`).

GOTCHA

If the agent is offline, verify the secret and controller URL in the agent container environment variables. Inside Docker Compose, the agent must reach the controller via `http://jenkins:8080` , not `localhost` .

INTERVIEW CHECK

Q: `down` vs `down -v` : which one wipes `jenkins_home` ?

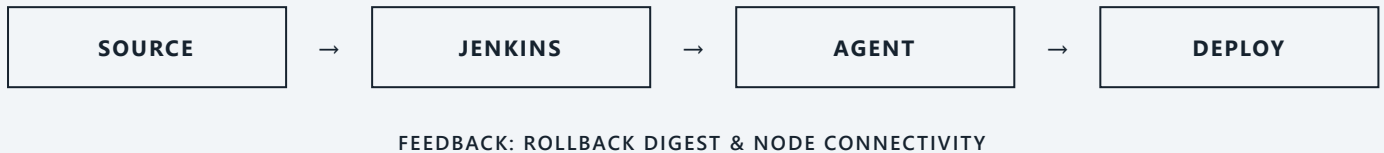
A: `docker compose down -v` wipes the volumes, deleting all Jenkins state, job history, and configurations. `down` alone stops containers but preserves the volume. This distinction is the difference between a clean reset and losing your CI system.

Threads: [Controllers & Agents](#)

Canonical Pipeline • Map

05 / System Map & Troubleshooting

The transition from cloud-native GitHub Actions to self-hosted Jenkins involves managing compute and execution paths directly. Failures shift from API permissions to network connectivity and node states.



Troubleshooting Flow

[ERROR] Agent offline in Jenkins UI

-> Recovery: Check agent container logs; verify secret & URL

[ERROR] Rollback fails (digest not found)

-> Recovery: Check registry retention policy; ensure old digests aren't deleted

[ERROR] Builds queue indefinitely

-> Recovery: Controller has 0 executors; assign label to agent

[WARN] Rollback breaks DB

-> Recovery: Data migrated forward; use expand-contract forward-fix

WHAT THIS PROVED

Recovery is a practiced path requiring immutable artifact retention. Jenkins architecture relies on a strict split between scheduling (controller) and execution (agent). General labs complete — the Jenkins track continues with shared libraries.

🎓 **EXAM** Broke production on purpose, then recovered by redeploying the previous known-good digest (**sha-a1b2c3d**) — no rebuild — and wrote a postmortem with a teeth-grade action item; stood up Jenkins as **controller plus docker-agent** with **zero executors** on the controller and ran the canonical **Build, Test, Deploy** pipeline on the agent label. Verified: production healthy in minutes, three green stages in Stage View, the controller ran zero builds. Recovery practiced, not improvised.

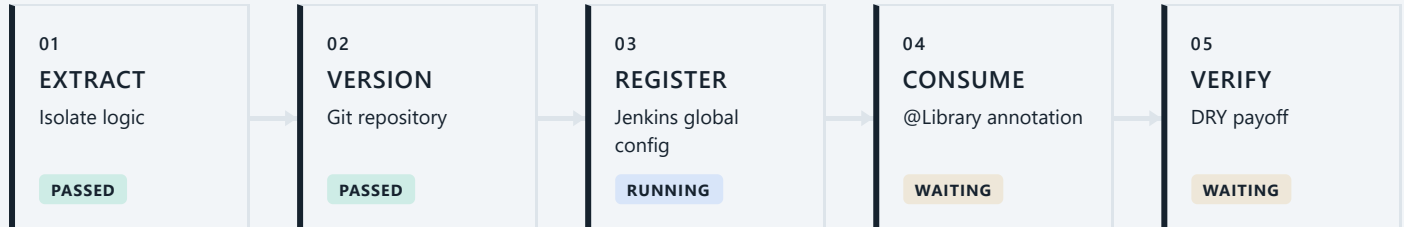
Next: Lab 07: Shared Library

Threads: [Controllers & Agents](#) • [Artifact Digest](#) • [Rollback & Recovery](#)

Lab 07: Shared Library

01 / Pipeline Code Reuse

As pipeline complexity grows, duplicating Groovy logic across dozens of Jenkinsfiles creates a maintenance nightmare. Extract a duplicated pipeline step into a versioned shared library consumed by multiple pipelines, achieving reuse without copy-paste.



OBJECTIVE

Move a shared step into a standalone Git repository, configure Jenkins to use it as a Global Pipeline Library, and call it from two distinct pipelines.

ARCHITECTURE

Library repo (`vars/standardTest.groovy`) → Jenkins Global Pipeline Libraries → multiple Jenkinsfiles call the same step → one fix propagates to all consumers.

Setup: The Library Repository

Create a repository named `pipeline-library`. Inside, create a directory named `vars` and a file `standardTest.groovy`. This file defines a custom step that can be called by any pipeline.

```
# File: vars/standardTest.groovy

def call(Map cfg = [:]) {

    sh "\${cfg.cmd}?: 'npm test'"

}
```

WHY IT MATTERS

Versioned libraries are dependencies. They are pinned, reviewed, and rollback-able. Copy-pasted steps are untrackable; shared libraries bring standard software engineering practices to pipeline code.

Threads: [Controllers & Agents](#)

Library Consumption

02 / Registration & Consumption

With the library code in a Git repository, Jenkins must be configured to trust and load it. Once registered, pipelines can import the library using an annotation.

Step 1: Register the Library

```
Manage Jenkins -> System -> Global Pipeline Libraries

> Library Name: my-lib

> Default Version: main

> Retrieval Method: Modern SCM

> Project Repository: https://github.com/user/pipeline-library.git
```

Step 2: Consume from Two Pipelines

In both Jenkinsfiles, import the library and call the custom step. The `_` in the annotation tells Jenkins to import the library but not run any global scripts automatically.

```
@Library('my-lib') _

pipeline {

  agent { label 'docker-agent' }

  stages {

    stage('Test') {

      steps { standardTest(cmd: 'npm test') }

    }

  }

}
```

- ☐ Both pipelines call the library step successfully.
- ☐ Breaking the library breaks both (proves sharing); reverting fixes both.
- ☐ Library change went through PR review on its own repo.

GOTCHA

If you encounter `Library my-lib not found`, check two things: the name in the Jenkinsfile matches the Global config exactly, and the default version (e.g., `main`) exists in the repository.

Threads: [Controllers & Agents](#)

Library Consumption · Map

03 / System Map & Troubleshooting

Shared libraries introduce external dependencies into the pipeline. Failures usually stem from untrusted code execution restrictions or branch resolution mismatches.



FEEDBACK: VERSIONED UPDATES PROPAGATE TO ALL CONSUMERS

Troubleshooting Flow

```
[ERROR] Library my-lib not found
```

```
-> Recovery: Check Global config name + repo branch existence
```

```
[ERROR] Script not allowed to use method
```

```
-> Recovery: Approve signatures in In-process Script Approval
```

```
[WARN] Stale logic executing
```

```
-> Recovery: Verify Default Version in Jenkins config matches intended branch
```

INTERVIEW CHECK

Q: Why must library changes go through PR review on their own repo?

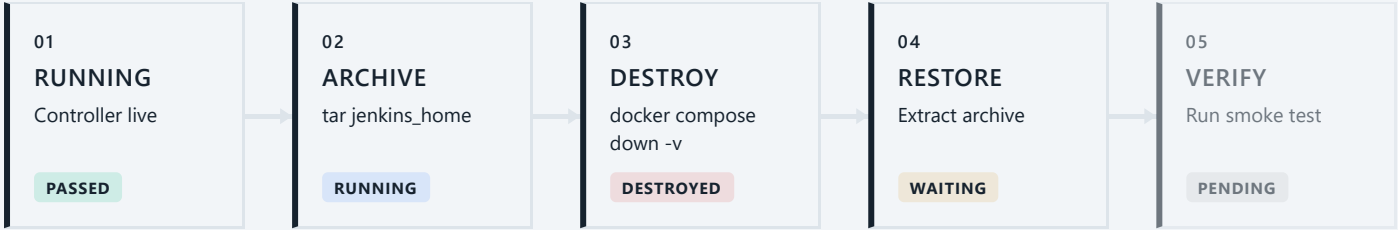
A: Unreviewed shared code risks propagating a breaking change or security vulnerability to all pipelines simultaneously. A PR review acts as a gatekeeper, ensuring the shared logic is safe before Jenkins consumers pull the new version.

Threads: [Approval Gates](#)

Lab 08: Backup & Restore

04 / Disaster Recovery

Backups are imaginary until tested by restoration. Prove disaster recovery: back up the Jenkins controller's home directory, destroy the controller entirely, restore on fresh infrastructure, and replay a smoke pipeline.



OBJECTIVE

Perform an offline backup of `JENKINS_HOME`, wipe the controller and volumes, restore the archive to a fresh controller, and verify job history and credentials.

OFFLINE BACKUP

An offline backup ensures a consistent snapshot. Copying files while Jenkins is live risks capturing half-written state, leading to corrupted configurations upon restore.

Pre-Flight Check

SETUP Note one job's last build number and one credential ID. These are the exact artifacts you will verify after the restore operation.

Threads: [Controllers & Agents](#)

Backup & Restore

05 / The Procedure

Execute the disaster recovery drill. The process involves stopping the container, archiving the volume, wiping the slate clean, and rebuilding from the archive.

Step 1: Back Up

```
$ docker compose stop jenkins

$ docker run --rm -v jenkins-lab_jenkins_home:/data \
  -v "$PWD":/out alpine tar czf /out/jenkins-backup.tar.gz -C /data .

jenkins-backup.tar.gz created successfully.

$ docker compose start jenkins
```

Step 2: Destroy and Restore

```
$ docker compose down -v # Wipe EVERYTHING

$ docker compose up -d # Fresh controller

$ docker compose stop jenkins

$ docker run --rm -v jenkins-lab_jenkins_home:/data \
  -v "$PWD":/in alpine tar xzf /in/jenkins-backup.tar.gz -C /data

$ docker compose start jenkins

Restore complete. Jenkins initializing...
```

Verification Checklist

- ☐ Job list + history identical (check the recorded build number).
- ☐ Credential ID present and usable.
- ☐ Smoke pipeline (from Lab 06) runs green on the restored controller.

GOTCHA

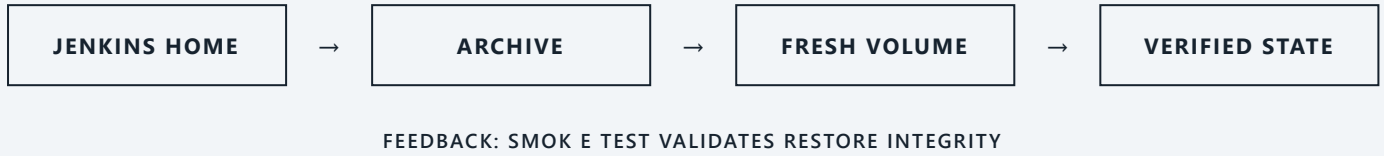
If you restore on a different Jenkins image tag, you may encounter plugin version drift. Always pin the `jenkins:1ts` image digest for both controller and backup to guarantee compatibility. If permission errors occur, ensure the volume is owned by UID 1000 (`chown -R 1000:1000`).

Threads: [Controllers & Agents](#)

Backup & Restore • Map

06 / System Map & Troubleshooting

Recovery is measured, not assumed. The disaster recovery drill exposes hidden dependencies on plugin versions and file system permissions that only surface during a real restore.



Troubleshooting Flow

[ERROR] Controller fails to start after restore

-> Recovery: Plugin version drift; pin image digest

[ERROR] Jobs missing or loading indefinitely

-> Recovery: Volume permissions; run `chown -R 1000:1000`

[WARN] Credential exists but fails to authenticate

-> Recovery: Re-enter secret text; secrets may not decrypt if master key changed

INTERVIEW CHECK

Q: What is the restore proof?

A: The restore proof is running the Lab 06 smoke pipeline on the restored controller. A green status verifies that the agent connection, job configuration, and execution environment were all recovered successfully from the archive.

WHAT THIS PROVED

Recovery is a practiced path. The Jenkins labs are complete. The system can be rebuilt from immutable artifacts and verified archives, closing the loop on CI/CD engineering.

🔑 **EXAM** Extracted duplicated logic into **vars/standardTest.groovy**, registered **my-lib** as a Global Pipeline Library, consumed it from two pipelines via **@Library('my-lib')** ; proved disaster recovery by archiving **JENKINS_HOME** offline, wiping everything with **docker compose down -v**, restoring the archive onto a fresh controller and replaying the smoke pipeline. Verified: the DRY payoff across pipelines, and job history plus credentials back after restore.

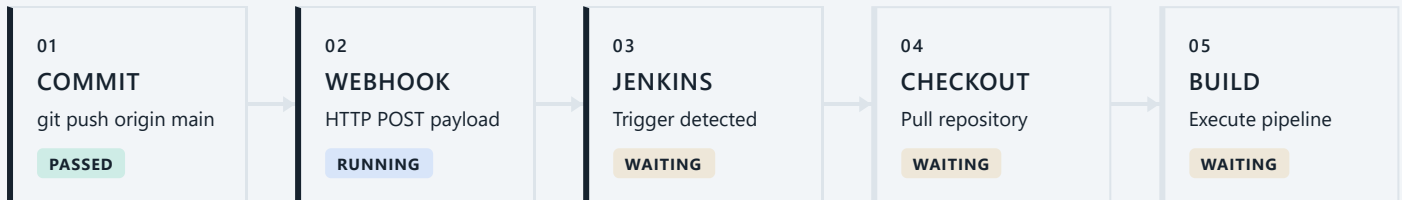
Next: Automated Triggering

Threads: [Artifact Digest](#) • [Controllers & Agents](#) • [Supply-Chain Security](#)

Automated Triggering

01 / Automated Integration Foundations

A pipeline is not magic. It is a deterministic sequence of machines making decisions about your code. However, for this sequence to be truly continuous, the transition between a developer's commit and the pipeline execution must be automatic. This is achieved through webhooks.



WHAT

A webhook is a custom HTTP callback triggered by an event, such as a git push. When the event occurs, the source system (GitHub) sends an HTTP POST request to the target system (Jenkins).

WHEN

Whenever a change is pushed to the repository. This includes commits to main, feature branches, or tagged releases, depending on the configured filter.

WHY

Polling (Jenkins constantly asking GitHub "any changes?") is inefficient and introduces latency. Webhooks provide instant, event-driven execution with zero wasted resources.

WHAT CHANGES

The development loop tightens. Feedback on a broken commit returns in minutes, not hours. The integration of code becomes a continuous, automatic flow rather than a manual chore.

WHY THIS EXISTS

Without automated triggering, a developer must manually notify the CI system that new code exists. This context switch breaks flow and inevitably leads to forgotten builds and integration delays. The webhook closes the gap between intent and execution.

Configuration Pre-Flight

02 / Pre-Flight Analysis

Before configuring a single setting, verify that all tools, credentials, and network paths are operational. A failure in webhook delivery is silent—Jenkins simply never runs. Resolve all dependencies in this phase before executing the setup.

Environment Verification

```
$ curl -s -o /dev/null -w "%{http_code}" http://localhost:8080

200

$ gh auth status

Logged in to github.com as dev-user

$ ngrok http 8080

Session Status      online

Forwarding          http://1a2b-3c4d.ngrok.io -> http://localhost:8080
```

Dependency Check

Requirement	Status	Action if Missing
Jenkins Admin Access	CONFIRMED	Request access from Jenkins admin
GitHub PAT (repo scope)	CONFIRMED	Generate in GitHub Developer Settings
Public Jenkins URL	CONFIRMED	Start ngrok or configure reverse proxy
Target Repository URL	CONFIRMED	Identify repo and default branch (main/master)

GOTCHA

If Jenkins is running locally (WSL, Docker, or localhost), GitHub cannot reach it. A tunneling tool like `ngrok` is mandatory to expose Jenkins to the public internet. Do not proceed to webhook configuration without a public URL.

Network Reachability

03 / Reachability & Network Paths

The webhook requires a clear HTTP path from GitHub's servers to your Jenkins instance. This is the most common point of failure in CI/CD automation. Determine your network topology before setting the Payload URL.



Establishing the Tunnel

If Jenkins is behind a NAT or local firewall, route traffic through a tunnel. This provides a temporary public URL that forwards incoming HTTP requests directly to your local port.

COMMAND

```
$ ngrok http 8080
```

RESULT

```
Forwarding  
https://1a2b-3c4d.ngrok.io  
→ http://localhost:8080
```

INTERVIEW CHECK

Q: Why can't I just use my local IP for the webhook?

A: GitHub's webhooks are sent from their public cloud infrastructure. Your local machine's IP (e.g., 192.168.x.x) is not routable from the public internet. A tunnel or public IP is required to bridge the connection.

Pipeline Construction

04 / Job & Credential Setup

Build the Jenkins pipeline structure and configure authenticated access to the GitHub repository. Never use raw GitHub passwords; always use a Personal Access Token (PAT) stored securely in Jenkins.

Step 1: Create Pipeline Job

```
Jenkins Dashboard -> New Item

> Name: github-webhook-pipeline

> Type: Pipeline

Pipeline Definition: Pipeline script

Initial Script:

pipeline {

    agent any

    stages { stage('Test') { steps { echo 'Build OK' } } }

}
```

Step 2: Configure Secure Credentials

Navigate to Pipeline Syntax -> checkout: Check out from version control. Generate the Groovy snippet using a new credential of type Username with password. Use your GitHub username as the username, and the PAT as the password.

GOTCHA

GitHub has deprecated password authentication for Git operations. Using your account password will result in a 403 Forbidden error during checkout. The PAT acts as a scoped, revocable proxy for your password.

INTERVIEW CHECK

Q: What is the difference between a PAT and a password?

A: A password grants full access to an account. A PAT is a scoped token that limits access to specific repositories and actions, can have an expiration date, and can be revoked without changing the account password.

Threads: [Controllers & Agents](#)

Lab 01: Authenticated Checkout

05 / Lab: Secure Checkout

Execute the authenticated checkout configuration. This lab verifies that Jenkins can securely access the private repository using the generated PAT credentials.

- OBSERVE**

The pipeline needs to pull source code from a private repository. Anonymous access is disabled. We need to verify the credential binding works.
- EXECUTE**

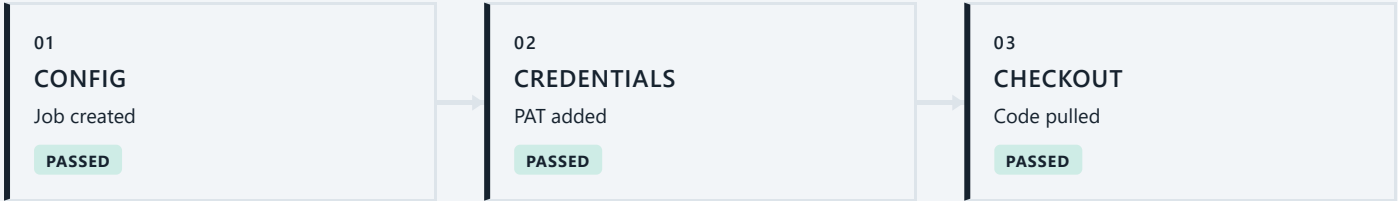
In Jenkins Job Configuration, paste the generated Groovy `checkout` snippet into the Pipeline script block. Ensure the `credentialsId` matches the ID of the created PAT credential (e.g., `github-token`).
- VERIFY**

Save the job and click "Build Now". Check the Console Output for the git fetch commands succeeding without authentication errors.
- EXPLAIN**

Jenkins injected the PAT into the Git command environment, allowing the fetch to succeed over HTTPS. The workspace now contains the repository files.

Generated Groovy Snippet

```
checkout([
    $class: 'GitSCM',
    branches: [[name: '*/main']],
    userRemoteConfigs: [[
        url: 'https://github.com/user/repo.git',
        credentialsId: 'github-token'
    ]]
])
```



The Webhook Bridge

06 / The Webhook Bridge

With the pipeline ready, establish the automated trigger. This requires configuring the webhook on GitHub and enabling the corresponding listener on Jenkins.

Architecture Flow



Step 5: GitHub Configuration

In GitHub, navigate to Repo Settings -> Webhooks -> Add webhook. Set the Payload URL to the Jenkins endpoint. The trailing slash is critical.

Payload URL:

```
https://github-webhook/
```

Content type: application/json

Trigger: Just the push event

GOTCHA

The URL `/github-webhook/` must include the trailing slash. Without it, Jenkins' routing will fail and return a 404, causing GitHub to mark the webhook as failed (red X).

Step 6: Jenkins Trigger Configuration

In the Jenkins Job configuration, scroll to **Build Triggers**. Enable **GitHub hook trigger for GITScm polling**. This tells Jenkins to react when the webhook payload is received, rather than polling on a schedule.

Lab 02: End-To-End Trigger

07 / Lab: End-to-End Trigger

Test the complete automation loop. A commit pushed to GitHub should automatically trigger the Jenkins pipeline without any manual intervention.

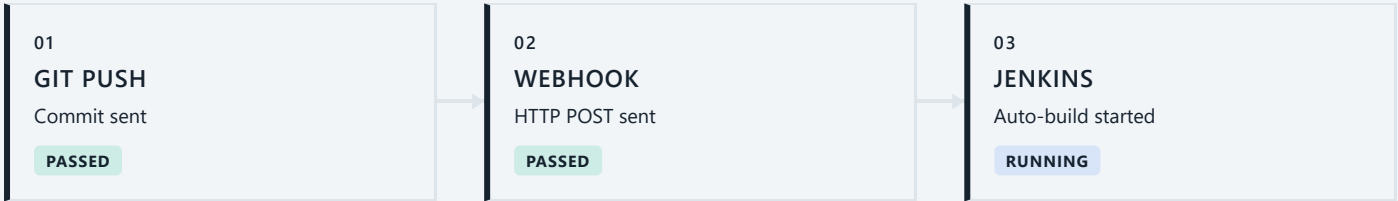
- OBSERVE** The webhook is configured, and the Jenkins trigger is enabled. We need to verify the event-driven connection.
- EXECUTE** Make a trivial change to the repository (e.g., add a comment to a file). Commit and push to the main branch.
- VERIFY** Watch the Jenkins dashboard. A new build should appear in the Build History within seconds, marked as triggered by "GitHub push".
- PROOF** The automation loop is closed. Human intervention is no longer required for integration. The pipeline is truly continuous.

COMMAND

```
$ echo "// update" >> index.html
$ git add .
$ git commit -m "test trigger"
$ git push origin main
```

RESULT

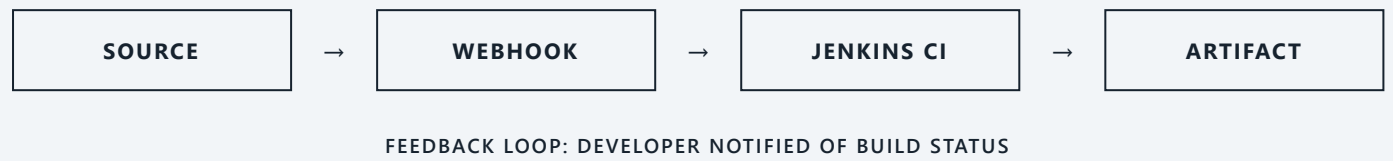
```
Enumerating objects: 5, done.
Writing objects: 100% (3/3), 340 bytes.
To github.com:user/repo.git
e7f8a9c..d1c2b3a main -> main
```



Lab 02 End-To-End Trigger • Map

08 / System Map & Troubleshooting

The automated trigger is a critical link in the broader deployment lifecycle. If it fails, the entire pipeline stalls. Use this map to contextualize the webhook and troubleshoot failures.



Troubleshooting Flow

Symptom	Root Cause	Recovery
Jenkins never triggers	Webhook not delivered	Check GitHub Webhook "Recent Deliveries" for HTTP errors
GitHub shows 404	Missing trailing slash	Ensure URL ends with <code>/github-webhook/</code>
Checkout fails (403)	Invalid PAT or expired	Regenerate PAT in GitHub, update Jenkins Credentials
Checkout fails (branch)	Wrong branch name	Verify default branch (<code>main</code> vs <code>master</code>)

DEV	Automated CI Trigger	PASSED
STAGING	Auto-Deploy on Success	RUNNING
PROD	Manual Approval Gate	BLOCKED

EXAM Built the event-driven loop: a PAT bound through the generated Groovy **checkout** snippet under **credentialsId: 'github-token'**, the **github-webhook-pipeline** job, and a GitHub webhook pointing at **/github-webhook/** with the trailing slash over an **ngrok http 8080** tunnel. Verified: pre-flight **200** from Jenkins, **gh auth status** clean, and a trivial push lands in Build History within seconds — no clicks.

Next: Container Delivery

Threads: [Approval Gates](#)

Container Delivery

01 / The Container Lifecycle

Moving from source code to a running application requires a deterministic path. The code must be compiled, packaged with its dependencies, and shipped as an immutable artifact. In modern CI/CD, this artifact is a Docker image.



WHAT

A pipeline that pulls source code, builds a Docker image, tags it with a unique build identifier, pushes it to a central registry, and optionally deploys it to a target environment.

WHEN

On every commit or merge to the main branch. The pipeline triggers automatically via webhook, executing the build and push sequence.

WHY

Containerizing the application ensures environment parity. The exact artifact that passed CI is the one deployed, eliminating "it works on my machine" discrepancies.

WHAT CHANGES

The release unit shifts from raw source code to a versioned, immutable container image. The registry becomes the source of truth for deployable software.

WHY THIS EXISTS

Without a standardized packaging format, deployment scripts must understand the application's dependency tree. A Docker image bundles the OS, libraries, and code into a single, static unit, radically simplifying the deployment target.

Threads: [Artifact Digest](#)

Build Architecture

02 / Build Architecture & Pre-Flight

The pipeline requires specific tools to be available on the Jenkins agent before execution. Verifying these dependencies upfront prevents silent failures during the Docker build or push stages.

Execution Architecture



Pre-Flight Verification

```
$ docker --version

Docker version 24.0.2, build cb74dfc

$ groups jenkins

jenkins : jenkins docker

$ git --version

git version 2.34.1
```

GOTCHA

Jenkins must have permission to execute Docker commands. The `jenkins` user must be added to the `docker` group, or the Docker Pipeline plugin must be configured to use a sidecar container. Otherwise, the build stage will fail with a "permission denied" socket error.

Threads: [Controllers & Agents](#)

Secure Authentication

03 / Registry Authentication

To push an image to a private or public Docker Hub repository, Jenkins must authenticate. Credentials must be stored securely in Jenkins and injected at runtime, never hardcoded into the Jenkinsfile.

Credential Configuration

In Jenkins, navigate to **Manage Jenkins** → **Credentials** and add a "Username with password" credential. Use your Docker Hub username and an Access Token (not your account password).

PIPELINE SNIPPET

```
withCredentials([
  usernamePassword(
    credentialsId: 'dockerhub-creds',
    usernameVariable: 'DOCKER_USER',
    passwordVariable: 'DOCKER_PASS'
  )
]) {
  sh 'echo $DOCKER_PASS | docker login -u $DOCKER_USER --password-stdin'
}
```

EXECUTION RESULT

```
[Pipeline] stage
[Pipeline] withCredentials
Masking supported pattern matches of $DOCKER_USER or $DOCKER_PASS
[Pipeline] sh
+ echo ****
Login Succeeded
```

GOTCHA

Docker Hub has deprecated password authentication for CLI push operations. You must generate a Personal Access Token (PAT) from Docker Hub's Account Settings → Security and use that token as the password in Jenkins.

INTERVIEW CHECK

Q: Why use `--password-stdin` instead of `-p <password>` ?

A: Using `-p` on the command line exposes the password in the process list and shell history, which other users on the system could read. `--password-stdin` pipes the token securely via standard input, preventing it from being logged.

Threads: [Controllers & Agents](#)

Pipeline Definition

04 / Jenkinsfile Construction

The declarative pipeline orchestrates the build, tag, and push sequence. Environment variables capture the image name and build tag, ensuring traceability between the Git commit, the Jenkins build number, and the Docker image.

```
pipeline {  
  
    agent any  
  
    environment {  
  
        IMAGE_NAME = "myusername/app"  
  
        IMAGE_TAG  = "build-${env.BUILD_NUMBER}"  
  
    }  
  
    stages {  
  
        stage('Code Checkout') { steps { checkout scm } }  
  
        stage('Docker Build') {  
  
            steps { sh "docker build -t ${IMAGE_NAME}:local ." }  
  
        }  
  
        stage('Image Tagging') {  
  
            steps { sh "docker tag ${IMAGE_NAME}:local ${IMAGE_NAME}:${IMAGE_TAG}" }  
  
        }  
  
        stage('Image Push') {  
  
            steps { sh "docker push ${IMAGE_NAME}:${IMAGE_TAG}" }  
  
        }  
  
    }  
  
}
```

Generated Artifact

myusername/app:build-123

WHY IT MATTERS

Tagging the image with `build-${BUILD_NUMBER}` creates an immutable, traceable link to the Jenkins execution. If a deployment fails, you can immediately pull the exact image that was deployed and inspect it. Using `:latest` destroys this traceability.

Threads: [Controllers & Agents](#) · [Artifact Digest](#) · [GitOps Pull Model](#)

Simulated Deployment

05 / The Deployment Track

Once the image is in the registry, the pipeline can optionally deploy it to a target environment. In this lab, the target is FLOCI, a simulated AWS infrastructure environment. The deployment stage must be conditional to allow for build-only runs.



Environment Lanes

BUILD	Docker Build & Push to Registry	PASSED
FLOCI	Run application container in simulated AWS	RUNNING

Conditional Deployment Stage

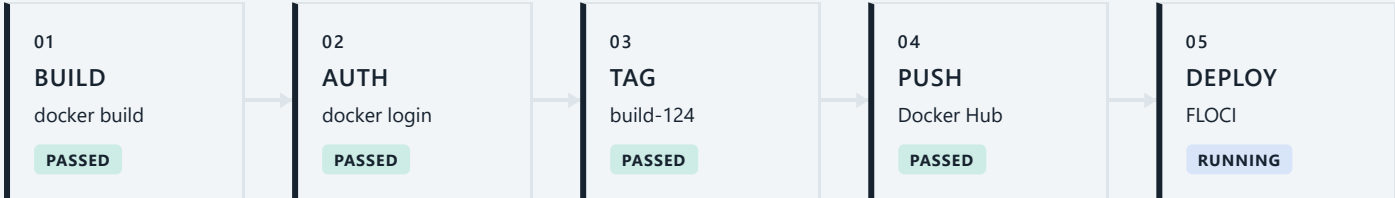
```
stage('Application Deployment') {  
  
    when { expression { return params.DEPLOY_TO_FLOCI == true } }  
  
    steps {  
  
        echo "Deploying ${IMAGE_NAME}:${IMAGE_TAG} to FLOCI..."  
  
        // FLOCI specific run command here  
  
    }  
  
}
```

Lab: End-To-End Delivery

06 / Lab: End-to-End Delivery

Execute the pipeline and verify the artifact progression from source code to registry deployment.

- OBSERVE** The repository contains the Dockerfile and Jenkinsfile. The webhook is configured. We need to verify the image is built and pushed successfully.
- EXECUTE** Trigger the pipeline via a Git push or manual "Build Now". Observe the Jenkins console output for each stage.
- VERIFY** Check Docker Hub for the new tag `build-<N>`. If FLOCI deployment is active, verify the container is running in the simulation.
- PROOF** The application is now packaged as an immutable artifact and delivered to the registry, ready for deployment.



COMMAND

```
$ docker push myusername/app:build-124
```

RESULT

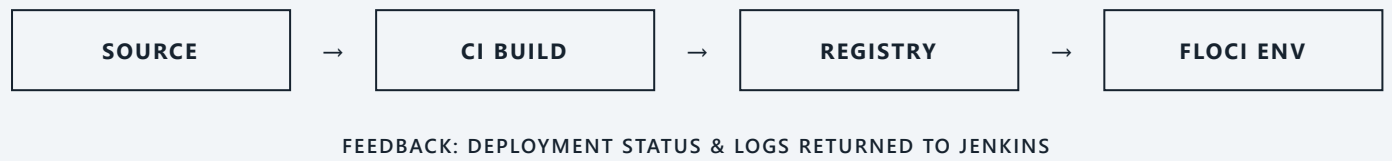
```
The push refers to repository
[docker.io/myusername/app]
2c3f4e5d6a7b: Pushed
build-124: digest: sha:4f5b... size: 528
```

Threads: [Artifact Digest](#) · [Controllers & Agents](#)

End-To-End Delivery · Map

07 / System Map & Troubleshooting

The container delivery pipeline is a critical path. Failures in building or pushing artifacts will block all downstream deployments. Use this map to contextualize the Docker build process and resolve common failure states.



Troubleshooting Flow

Symptom	Root Cause	Recovery
Build fails (permission)	Jenkins user not in docker group	<code>sudo usermod -aG docker jenkins</code>
Push fails (unauthorized)	Invalid Docker Hub token	Regenerate Access Token, update Jenkins Credentials
Deploy fails (not found)	Image tag mismatch	Ensure FLOCI pulls <code>build-\${BUILD_NUMBER}</code>
Build slow/bloated	Invalid Dockerfile layer cache	Optimize Dockerfile (copy deps first)

INTERVIEW CHECK

Q: Why tag with the build number instead of the Git commit hash?

A: While commit hashes are immutable, they are not human-readable or sequential. The build number provides a simple, incrementing identifier that directly maps to the Jenkins execution, making it easier to discuss and trace ("build 124 failed").

🔑 EXAM

Built the container path: **IMAGE_NAME** with **IMAGE_TAG** of `build-${BUILD_NUMBER}`, **docker build** then **docker tag** then **docker push**, registry login via `--password-stdin` (never `-p <password>`), and an optional **FLOCI** deploy stage gated by **when** on **DEPLOY_TO_FLOCI**. Verified: the new tag **build-<N>** appears in Docker Hub, the container runs in simulation when enabled, the image stays in the registry when not.

Next: [Multibranch Pipelines](#)

Threads: [Artifact Digest](#)

Multibranch Pipelines

01 / Multibranch Foundations

A standard pipeline executes a single predefined script. A Multibranch Pipeline is structurally different: it is a folder of individual pipeline jobs, one per discovered branch. Jenkins scans the repository, finds branches, and looks for a Jenkinsfile to build with.



WHAT

A Jenkins job type that automatically creates a sub-job for every branch in a Git repository containing a Jenkinsfile.

WHY

It eliminates the need to manually create and configure a new Jenkins job every time a developer creates a new branch.

WHEN

On a periodic schedule (e.g., every 5 minutes) or manually triggered. This task deliberately ignores webhooks to simulate an air-gapped environment.

WHAT CHANGES

The relationship between source code and CI inverts. The repository defines the jobs, not Jenkins. Branch deletion leads to job orphaning.

WHY THIS EXISTS

In dynamic environments with feature branches, manually managing Jenkins jobs is impossible. Multibranch pipelines push job creation down into the repository itself, making the CI configuration version-controlled and branch-specific.

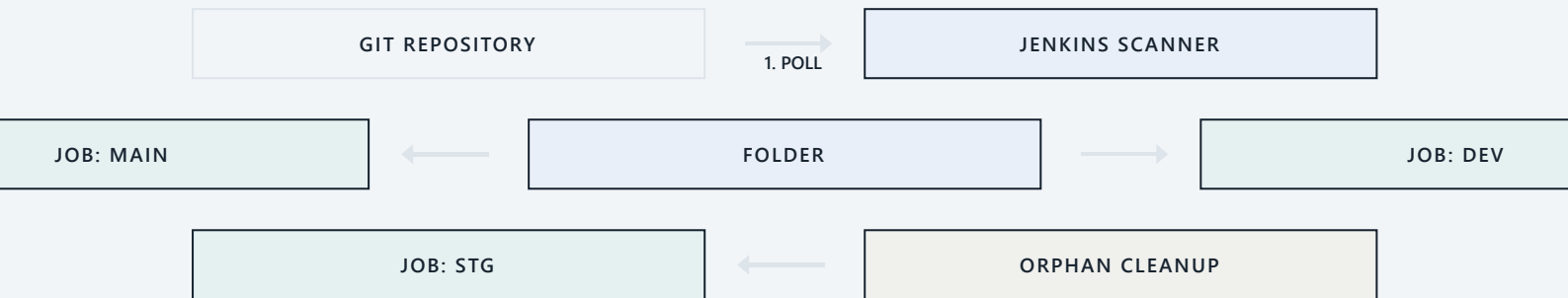
Threads: [Controllers & Agents](#)

Scan Architecture

02 / Architecture & Pre-Flight

The Multibranch Pipeline relies on the plain Git branch source, not GitHub-specific integration. This ensures compatibility with any Git provider and works perfectly in air-gapped or firewall-restricted networks where inbound webhooks are blocked.

Execution Architecture



Pre-Flight Verification

```
$ git --version

git version 2.34.1

$ git ls-remote https://github.com/user/repo.git

e7f8a9c1d2e3f4g5h6i7j8k9l0m1n2o3      HEAD
e7f8a9c1d2e3f4g5h6i7j8k9l0m1n2o3      refs/heads/main
a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6      refs/heads/dev
f1e2d3c4b5a69870123456789abcdef0      refs/heads/stg

Access confirmed.
```

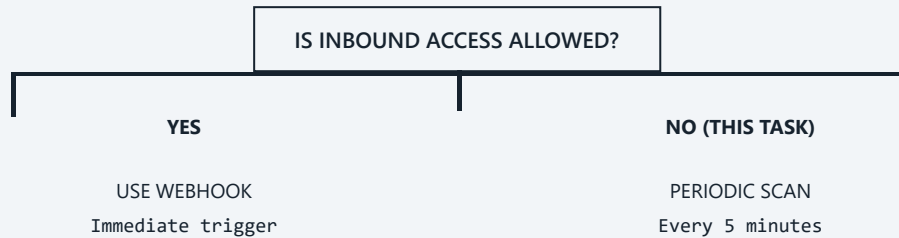
GOTCHA

Individual branch jobs inside the Multibranch folder (e.g., `main`, `dev`) are **not directly configurable**. All behavior (branch filtering, checkout, scan triggers) is controlled from the parent Multibranch Pipeline job's configuration only.

Job Configuration

03 / Discovery & Filtering

Configure the Multibranch Pipeline job to use a plain Git source. Filter discovery to exactly three branches: `main`, `dev`, and `stg`. Since no webhooks are used, rely on periodic scanning.



Branch Source Configuration

```
Jenkins -> New Item -> Multibranch Pipeline

> Branch Source: Git (Plain Git, not GitHub)

> Project Repository: https://github.com/user/repo.git

> Behaviors -> Filter by name (with wildcards)

> Include: main dev stg

> Exclude: (blank)

> Additional Behaviors:

> - Check out to matching local branch

> - Clean before/after checkout

> Scan Multibranch Pipeline Triggers: Every 5 minutes
```

GOTCHA

Ensure "Check out to matching local branch" is enabled. Without it, Jenkins checks out a detached commit. While builds may succeed, Git metadata within the pipeline won't reflect the actual branch name, breaking `when { branch 'stg' }` conditionals.

Pipeline Logic

04 / Conditional Execution

A single shared Jenkinsfile must adapt its behavior based on the branch it is building. The `env.BRANCH_NAME` variable, injected by the Multibranch plugin, enables this via `when` conditionals.

```
pipeline {  
    agent any  
  
    stages {  
        stage('Build') { steps { echo 'Building...' } }  
  
        stage('Staging Verification') {  
            when { expression { return env.BRANCH_NAME == 'stg' } }  
  
            steps { sh 'cat README.md' }  
        }  
    }  
}
```

Execution Lanes by Branch

MAIN	Build Staging Verification (SKIPPED by when)
DEV	Build Staging Verification (SKIPPED by when)
STG	Build Staging Verification (cat README.md)

INTERVIEW CHECK

Q: How does a single Jenkinsfile adapt to different branches?

A: The Multibranch plugin injects `env.BRANCH_NAME`. The `when` directive evaluates this variable before executing a stage, allowing branch-specific logic (like deployment or extra tests) to reside safely in the same file.

Threads: [Controllers & Agents](#)

Lab: Lifecycle Management

05 / Lab: Orphan Cleanup

Branches are not permanent. When a branch is deleted upstream in Git, the corresponding Jenkins job must be removed to prevent clutter. This process does not happen instantly; it requires a scan.

OBSERVE A feature branch `test-branch` was merged and deleted from the remote Git repository. The Jenkins folder still shows the job.

EXECUTE Trigger a manual scan ("Scan Multibranch Pipeline Now") or wait for the periodic scan to execute.

VERIFY Check the scan log. It will report `test-branch` as an orphaned item. The job will be removed from the Jenkins folder view.

PROOF Jenkins state perfectly mirrors the Git repository state. No stale jobs remain.

Folder State Transition

```
└─ multibranch-job SCANNED
   └─ main ACTIVE
   └─ dev ACTIVE
   └─ stg ACTIVE
   └─ test-branch ORPHANED
```

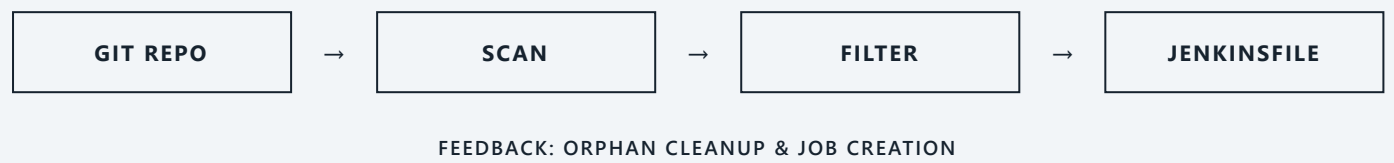
GOTCHA

Orphaned branch cleanup only happens **after a rescan**. Deleting a remote branch does not immediately remove its Jenkins job. The next scan evaluates the repository state and removes orphaned items.

Lifecycle Management • Map

06 / System Map & Troubleshooting

The Multibranch Pipeline acts as an automated job manager. Understanding its scan and filter logic is critical for resolving issues where branches are missed or jobs are not created.



Troubleshooting Flow

Symptom	Root Cause	Recovery
Branch not discovered	Filter mismatch	Check "Include" wildcard in Branch Source config
Branch found, no job	Jenkinsfile missing	Commit a Jenkinsfile to the root of that branch
when condition fails	Detached HEAD checkout	Enable "Check out to matching local branch"
Stale jobs remain	No scan executed	Trigger manual scan or wait for periodic interval

INTERVIEW CHECK

Q: Why use plain Git branch source instead of GitHub-specific integration?

A: The plain Git source works with any Git provider (GitLab, Bitbucket, etc.) and functions in air-gapped environments where inbound webhooks are blocked. It relies on periodic polling, prioritizing compatibility over immediate trigger speed.

🦋 EXAM

Built a Multibranch Pipeline on the plain **Git** branch source discovering exactly **main, dev, stg** on a periodic scan, one shared Jenkinsfile adapting per branch via **env.BRANCH_NAME when** conditionals, and orphan cleanup on scan. Verified: three sub-jobs, the stg-only stage skipped elsewhere, a deleted branch removed after rescan — Jenkins state mirrors Git.

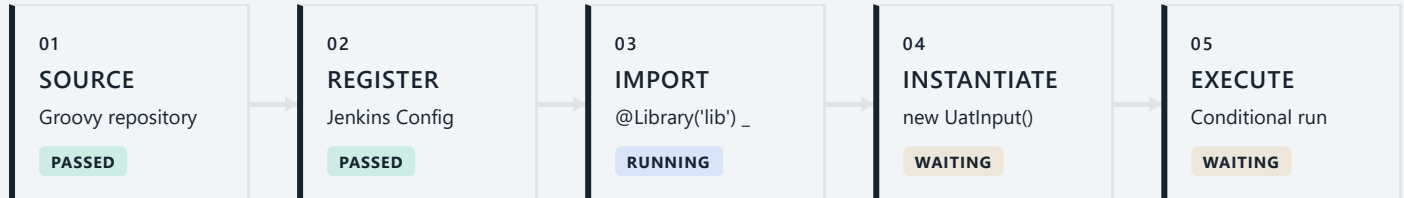
Next: Shared Libraries

Threads: Controllers & Agents

Shared Libraries

01 / Reusable Pipeline Logic

As pipeline complexity grows, duplicating Groovy logic across dozens of Jenkinsfiles creates a maintenance nightmare. A Shared Library allows you to store versioned Groovy classes in a standard Git repository and import them into any pipeline.



WHAT

A Git repository containing Groovy code that Jenkins dynamically downloads and makes available to pipelines via an import statement.

WHY

To enforce DRY (Don't Repeat Yourself) principles in CI/CD. Updates to shared logic propagate to all pipelines without modifying individual Jenkinsfiles.

WHEN

When multiple pipelines share common logic: deployment steps, approval gates, metric parsing, or custom build wrappers.

WHAT CHANGES

Pipelines become thinner. Complex business logic is abstracted into testable Groovy classes, moving Jenkins configuration closer to standard software engineering.

WHY THIS EXISTS

Without shared libraries, organizations end up with hundreds of bloated Jenkinsfiles that are impossible to update. A shared library transforms CI/CD logic into a versioned, testable software dependency.

Threads: [Controllers & Agents](#)

Library Structure

02 / Repository Architecture

Jenkins expects a specific directory structure for shared libraries. Groovy classes must reside in the `src/` directory, following standard Java package naming conventions.

Directory Structure

```
jenkins-shared-library
├── src
│   ├── com
│   └── yourname
│       └── UatInput.groovy
```

Class Definition: UatInput.groovy

```
package com.yourname

class UatInput {

    def buildIsUatApproved() {

        def file = new File('tools/approved.txt')

        return file.exists()

    }

}
```

Execution Architecture



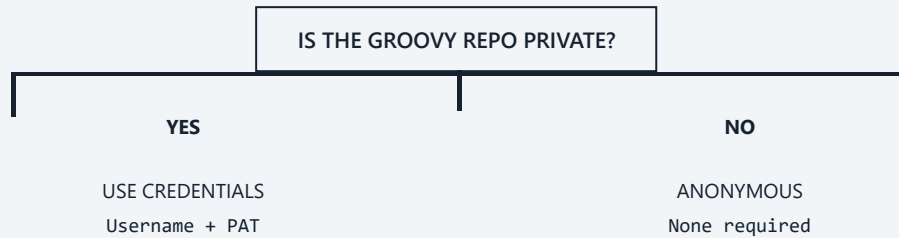
GOTCHA

The package path in the file (`package com.yourname`) must exactly match the folder structure (`src/com/yourname/`). A mismatch will result in a `ClassNotFoundException` during pipeline execution.

Global Configuration

03 / Jenkins Registration

Before a pipeline can import the library, Jenkins must be aware of its existence. Register the library globally so it is available to all jobs without per-job configuration.



System Configuration

Manage Jenkins -> Configure System

Global Pipeline Libraries -> Add

> Library Name: shared-library

> Default Version: master

> Retrieval Method: Modern SCM

> SCM: GitHub

> Project Repository: <https://github.com/user/repo.git>

Click "Validate" to test connection.

GOTCHA

By default, shared library scripts run in the Groovy Sandbox. If your custom class uses standard Java I/O (like `new File()`), you must uncheck "Use Groovy Sandbox" in the Pipeline job configuration, or explicitly approve the method signatures in **Manage Jenkins -> In-process Script Approval**.

INTERVIEW CHECK

Q: What is the difference between Global Pipeline Libraries and Automatic Pipeline Libraries?

A: Global libraries are available to any pipeline but must be explicitly imported via `@Library`. Automatic libraries are implicitly imported into all pipelines without the annotation, which is convenient but can hide dependencies.

Threads: [Controllers & Agents](#)

Scripted Execution

04 / Scripted Pipeline Usage

Import the library into a scripted pipeline using the `@Library` annotation. Instantiate the class and call its methods to evaluate workspace state.

```
@Library('shared-library') _  
  
import com.yourname.UatInput  
  
node {  
  
    stage('Check Approval') {  
  
        def uatInput = new UatInput()  
  
        def isApproved = uatInput.buildIsUatApproved()  
  
        println "Build is UAT approved: ${isApproved}"  
  
    }  
  
}
```

Execution Result (File Present)

COMMAND

```
$ touch tools/approved.txt  
$ build now
```

RESULT

```
[Pipeline] stage  
[Pipeline] { (Check Approval)  
Build is UAT approved: true  
[Pipeline] }
```

WHY IT MATTERS

The underscore `_` in `@Library('shared-library') _` is required when importing classes. It tells Jenkins to import the library but not automatically run any global variables or scripts, only making the classes available for explicit instantiation.

Declarative Execution

05 / Declarative & Conditionals

In declarative pipelines, the imported class can be used inside a `when` block to conditionally skip or execute stages based on workspace state.

```
@Library('shared-library') _

import com.yourname.UatInput

def uatInput = new UatInput()

pipeline {
    agent any

    stages {
        stage('Deploy to Prod') {
            when { expression { return uatInput.buildIsUatApproved() } }

            steps { echo 'Deployment approved.' }
        }
    }
}
```

State Transition

APPROVED

tools/approved.txt exists

RUN STAGE

UNAPPROVED

tools/approved.txt missing

SKIP STAGE

GOTCHA

The `when` directive is evaluated before the `agent` is allocated for the stage if `beforeAgent true` is not set. If your Groovy class reads files from the workspace, ensure the agent is checked out first, or evaluate the condition inside a `script` block within the `steps`.

Threads: [Controllers & Agents](#)

Lab: Conditional Validation

06 / Lab: End-to-End Validation

Test the shared library logic by toggling the existence of the approval file and observing the pipeline's state transition.

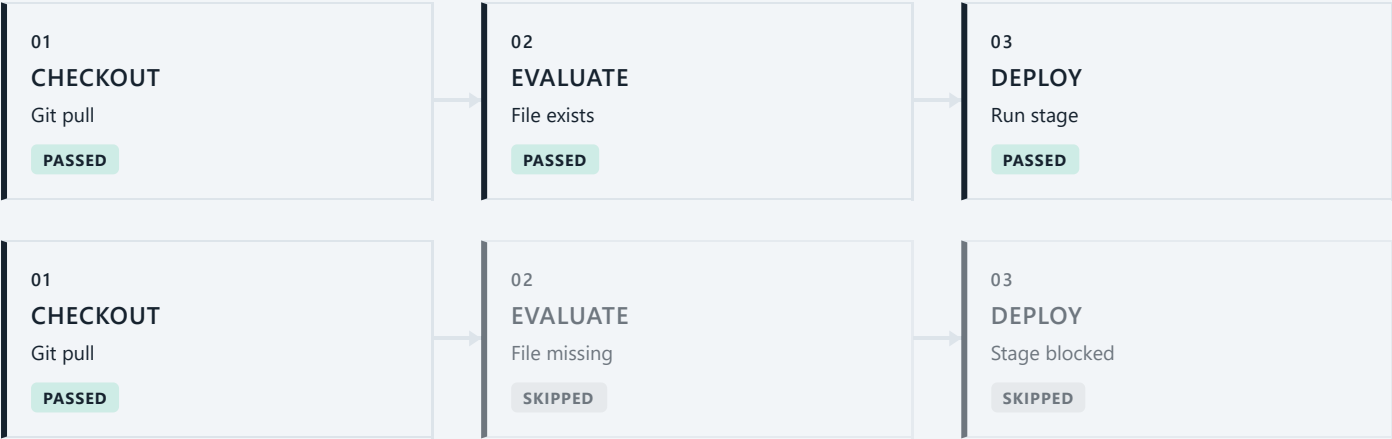
- OBSERVE**

The declarative pipeline is configured. We need to verify that the stage execution is strictly controlled by the UatInput class.
- EXECUTE**

Run the pipeline with `tools/approved.txt` present. Then, delete the file and run the pipeline again.
- VERIFY**

Check the Jenkins build logs. The first run should execute the stage. The second run should show "Stage skipped due to when conditional".
- PROOF**

The pipeline dynamically altered its behavior based on workspace state evaluated by external, versioned Groovy code.

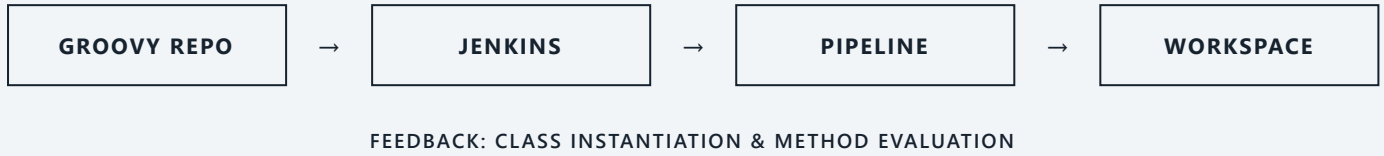


Threads: [GitOps Pull Model](#) · [Controllers & Agents](#)

Conditional Validation • Map

07 / System Map & Troubleshooting

Shared libraries introduce external dependencies into the pipeline. Failures usually stem from package path mismatches, sandbox restrictions, or network access to the Git repository.



Troubleshooting Flow

Symptom	Root Cause	Recovery
ClassNotFoundException	Package path mismatch	Ensure folder structure matches <code>package</code> declaration
Sandbox Rejection	File I/O blocked by Jenkins	Disable Groovy Sandbox or approve signatures
Library not found	Typo in <code>@Library</code> name	Match annotation to Global Library name exactly
Stale logic executing	Caching / wrong branch	Verify Default Version in Jenkins config

INTERVIEW CHECK

Q: Why disable the Groovy Sandbox for shared libraries?

A: The sandbox limits dangerous operations (like file system access) for untrusted scripts. Shared libraries are considered trusted code. Disabling the sandbox allows the library to perform necessary system operations, though it requires strict review of the library code.

🔑 **EXAM** Built a versioned Groovy library (`src/com/yourname/UatInput.groovy`, `buildIsUatApproved()` checking `tools/approved.txt`), registered **shared-library** globally, imported it with `@Library('shared-library')` _ in scripted and declarative pipelines with a **when** gate on Deploy to Prod. Verified: file present and the stage runs; file deleted and the log reads "Stage skipped due to when conditional". Behavior flips with workspace state.

Next: Infrastructure Pipeline

Threads: [Approval Gates](#)

Infrastructure Pipeline

01 / The Infrastructure Pipeline

Infrastructure as Code (IaC) requires the same rigorous CI/CD practices as application code. However, unlike application deployments, infrastructure changes can cause immediate, irreversible state shifts. The pipeline must therefore include strict validation and manual approval gates before applying changes.



WHAT

A Jenkins pipeline that executes Terraform commands against a simulated AWS environment (FLOCI) to provision VPCs, subnets, and compute resources.

WHEN

Triggered automatically by a GitHub webhook on push, but pauses for manual approval before the destructive `apply` stage.

WHY

To practice infrastructure automation safely. FLOCI mimics the AWS API without incurring real costs, allowing teams to test CI/CD workflows aggressively.

WHAT CHANGES

Infrastructure moves from manual CLI execution to a versioned, peer-reviewed, automated pipeline. State shifts become deliberate, logged events.

WHY THIS EXISTS

Without a pipeline, infrastructure drifts. Engineers apply changes locally, state files diverge, and recovery becomes impossible. A pipeline enforces a single, auditable path from code to cloud.

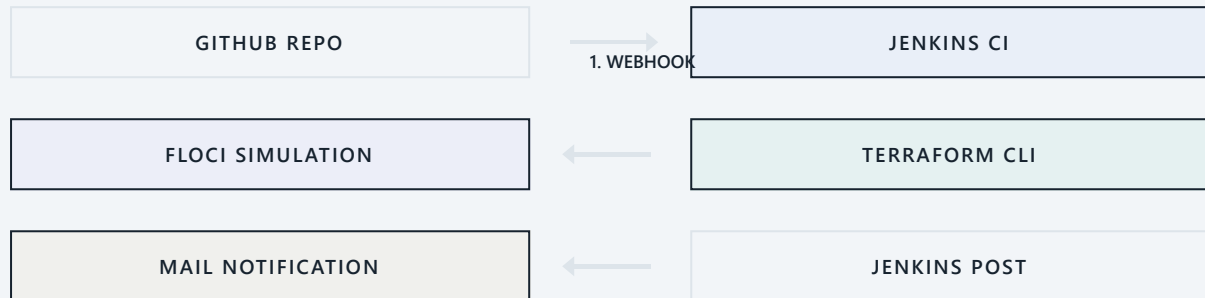
Threads: [Approval Gates](#) · [Controllers & Agents](#)

Provisioning Architecture

02 / Architecture & Pre-Flight

The pipeline bridges GitHub, Jenkins, and FLOCI. The critical integration point is the `Provider.tf` file, which redirects standard AWS API calls to the FLOCI simulation endpoint.

Execution Architecture



Pre-Flight Verification

```
$ terraform -version

Terraform v1.5.0

$ curl -s -o /dev/null -w "%{http_code}" http://localhost:8080

200

$ curl -s -o /dev/null -w "%{http_code}" http://floci-sim.local

200
```

GOTCHA

FLOCI is a simulation, not real AWS. Standard AWS credential patterns (IAM roles, `~/.aws/credentials`) do not apply. The `Provider.tf` must be explicitly configured with the FLOCI endpoint URL and simulation tokens gathered in Step 0.

Environment Isolation

03 / Workspace Isolation

Infrastructure state must be isolated per environment. A change in `dev` must never alter `prod` state. Terraform workspaces combined with environment-specific `.tfvars` files achieve this within a single pipeline.

Environment Tracks

DEV	Workspace: dev File: dev.tfvars	ACTIVE
STG	Workspace: stg File: stg.tfvars	IDLE
PROD	Workspace: prod File: Prod.tfvars	LOCKED

Workspace Selection Logic

```
sh 'terraform workspace select ${ENV_NAME} || terraform workspace new ${ENV_NAME}'

Switched to workspace "dev".
```

INTERVIEW CHECK

Q: Why use workspaces instead of separate directories for state?

A: Workspaces allow the same Terraform configuration to manage multiple distinct state files without duplicating code. It simplifies the pipeline by parameterizing the environment via a simple `--workspace` flag rather than altering backend paths dynamically.

Manual Approval Gate

04 / The Approval Gate

Terraform `apply` is destructive. A pipeline that auto-applies infrastructure changes is dangerous. The pipeline must pause after the `plan` stage, presenting the execution plan for human review before proceeding.



Jenkins Input Step

```
stage('Approve') {  
    steps {  
        input message: 'Apply this Terraform plan?', ok: 'Approve'  
    }  
}
```

GOTCHA

If the pipeline is aborted at the `input` step, Jenkins marks the build as `ABORTED`, not `FAILED`. Ensure your `post` actions distinguish between failure (requiring a mail alert) and abort (a deliberate human choice).

WHY IT MATTERS

Automation stops here because a human decision is required. The `plan` output is the contract; the `apply` is the execution. Skipping this gate removes accountability from the infrastructure lifecycle.

Threads: [Approval Gates](#)

Simulated Cloud

05 / FLOCI Provider Configuration

To target FLOCI instead of real AWS, the `Provider.tf` must override the default AWS endpoints. This intercepts the Terraform AWS provider's HTTP calls and routes them to the local simulation.

```
# Provider.tf

provider "aws" {

  region = "us-east-1"

# Redirect to FLOCI Simulation

  endpoints {

    ec2      = "http://floci-sim.local/ec2"

    s3       = "http://floci-sim.local/s3"

    elasticloadbalancing = "http://floci-sim.local/elb"

  }

  skip_credentials_validation = true

  skip_metadata_api_check    = true

  skip_requesting_account_id = true

}
```

COMMAND

```
$ terraform init
```

RESULT

```
Initializing the backend...
Initializing provider plugins...
Terraform has been successfully initialized!
```

WHY IT MATTERS

By skipping credential validation and overriding endpoints, Terraform operates in a sandbox. This allows engineers to test complex VPC topologies and EC2 deployments against a mock API, ensuring the IaC logic is sound before touching a real cloud billing account.

Lab: Provisioning & Recovery

06 / Lab: End-to-End Provisioning

Execute the pipeline and observe the state transitions, from webhook trigger through manual approval to simulated deployment and failure recovery.

- OBSERVE**

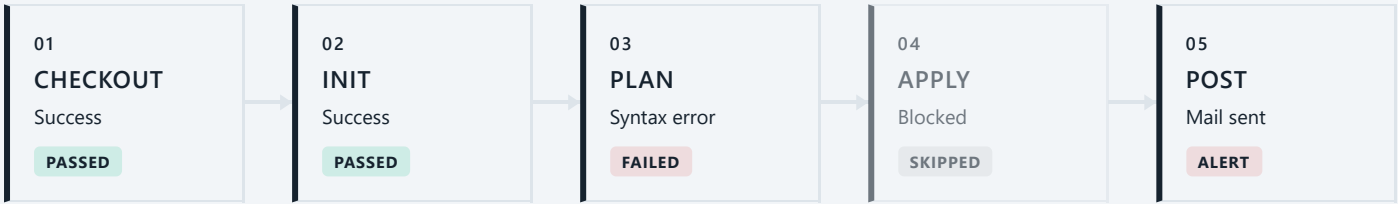
The GitHub repo contains the Terraform code and Jenkinsfile. The webhook is active. We need to verify the pipeline reaches the approval gate.
- EXECUTE**

Push a minor change to trigger the pipeline. Watch the stages: Checkout -> Init -> Workspace Select -> Plan. The pipeline will pause at "Approve".
- VERIFY**

Review the `plan` output in the Jenkins logs. Click "Approve". Confirm the `apply` stage runs and FLOCI reflects the new VPC resources.
- PROOF**

Introduce a syntax error in `VPC.tf` and push. The pipeline will fail at `Plan`. The `post { failure { mail ... } }` block executes, sending an alert.

Failure & Recovery Flow

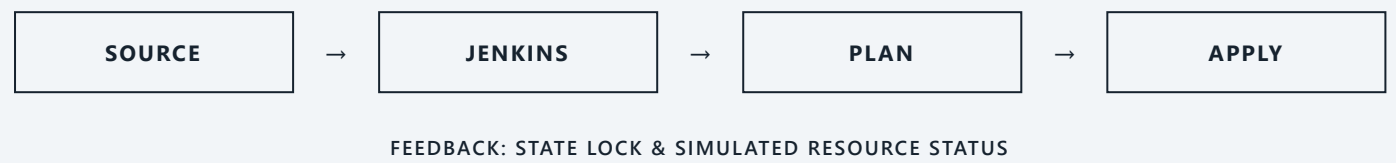


Threads: [Approval Gates](#) · [Controllers & Agents](#)

Provisioning & Recovery · Map

07 / System Map & Troubleshooting

Infrastructure pipelines fail in predictable ways: state conflicts, provider mismatches, or unauthorized credentials. Use this map to isolate the failure domain.



Troubleshooting Flow

Symptom	Root Cause	Recovery
Init fails (provider)	FLOCI endpoint unreachable	Verify FLOCI service is running and URL in Provider.tf
Plan fails (state lock)	Concurrent pipeline runs	Force unlock state or disable concurrent builds
Apply fails (403)	FLOCI simulation token invalid	Update Jenkins credentials for FLOCI provider
Pipeline paused forever	Approval gate ignored	Review Jenkins input prompt or abort build

INTERVIEW CHECK

Q: Why use FLOCI instead of a real AWS Sandbox account?

A: Real AWS sandboxes still incur costs and carry the risk of accidental production impact if IAM boundaries are misconfigured. FLOCI provides a zero-cost, zero-risk API mock that is ideal for testing the CI/CD pipeline mechanics themselves.

🔪 EXAM Built the infrastructure pipeline: Terraform against **FLOCI** via **Provider.tf** endpoint overrides, per-environment isolation with **terraform workspace select/new** plus **.tfvars** files, **Plan** then a human **input Approve** gate before **apply**, with post notifications. Verified: the pipeline pauses at Approve, the plan is reviewed in the logs, apply provisions the VPC in simulation — destructive changes never auto-apply.

Threads: [Approval Gates](#)



Quick Revision — Terms Table

Hermetic build	Same result in any environment; the runner uses no ambient machine state.
Lockfile	package-lock.json pins every dependency; keys the cache and makes npm ci exact.
DAG / needs	needs: build orders jobs; test never starts before build finishes.
Branch protection	main requires the green build check; merging on red is blocked.
Status check	The green check on a PR proving that commit passed the pipeline.
Immutable digest	sha-256 pin of exact image bytes; staging and production run one digest.
GHCR	Registry holding versioned images; pushes need packages: write.
SBOM (SPDX)	Ingredient list via anchore/sbom-action; answers the next CVE in minutes.
Promotion	Same digest staging to production; staging auto-deploys, production needs approval.
Approval gate	Human input step before apply or deploy; an abort reads ABORTED, not FAILED.

Rollback / forward-fix	Redeploy the old digest with no rebuild; a migrated DB forces expand-contract.
Controller / agent	Brain schedules, muscle executes; zero executors on the controller, label docker-agent.
Multibranch	Folder of per-branch jobs discovering main, dev, stg on a periodic scan.
Orphaned job	Job of a deleted branch, removed on rescan so Jenkins mirrors Git.
Shared library	Versioned Groovy repo (vars/, src/); imported with @Library; DRY pipelines.
JENKINS_HOME	Controller state; offline tar backup, wipe, restore, then a smoke-pipeline proof.
Webhook	Push fires a POST to /github-webhook/ with trailing slash; ngrok tunnels local Jenkins.
PAT	Token stored as github-token credentialsId; never a raw account password.
FLOCI	Simulated cloud targeted via Provider.tf endpoint overrides; zero real cost.
Workspace	terraform workspace select/new plus .tfvars isolates dev, stg and prod state.

KEY IDEA Cover the right column and define each term aloud. Any stumble goes back to its lab.

Threads: [Controllers & Agents](#) · [Artifact Digest](#) · [Hermetic Builds](#)



Concept Questions — Distilled Answers

Q1 What does the green check actually prove?

That this exact commit passed the checks in that pipeline — not that the app is correct in production.

Q2 What does needs: build guarantee?

Ordering plus isolation: the test job never starts before build finishes, and failures point at one stage.

Q3 Why is redeploying :latest dangerous?

A tag is a movable pointer; only the immutable digest proves staging and production run identical bytes.

Q4 Rollback or forward-fix — which one?

No data migration: redeploy the previous known-good digest with no rebuild. Migrated data: expand-contract forward.

Q5 Why zero executors on the controller?

Builds on the controller risk crashing all of CI; the controller schedules and agents execute.

Q6 Why --password-stdin instead of -p?

-p exposes the secret in the process list and shell history; stdin keeps it out of both.

Q7 The webhook is set but Jenkins never fires. First check?

The trailing slash on /github-webhook/ plus reachability: local Jenkins needs the ngrok tunnel up.

Q8 Why workspaces plus .tfvars for environments?

One configuration, isolated state per environment: select-or-create keeps dev, stg and prod from touching.

Threads: [Controllers & Agents](#) · [Artifact Digest](#) · [Rollback & Recovery](#)



Command Cheat Sheet

Node CI loop

<code>npm ci</code>	clean install from the lockfile
<code>npm test</code>	run the suite on the runner
<code>npm ci && npm test</code>	local hermetic repro of CI
<code>needs: build</code>	order test after build in the DAG

Compose / Jenkins

<code>docker compose up -d</code>	start controller plus agent
<code>docker compose stop jenkins</code>	halt controller for a consistent backup
<code>docker compose down -v</code>	wipe EVERYTHING including volumes
<code>chown -R 1000:1000</code>	fix volume ownership after restore
<code>sudo usermod -aG docker jenkins</code>	grant Docker rights to the agent user

Git / webhook / verify

<code>ngrok http 8080</code>	public tunnel to local Jenkins
<code>curl -s -o /dev/null -w "%{http_code}" http://localhost:8080</code>	pre-flight: expect 200
<code>gh auth status</code>	verify GitHub identity
<code>git ls-remote https://github.com/user/repo.git</code>	verify repo reachability plus branches
<code>/github-webhook/</code>	payload path incl. trailing slash

Delivery / Terraform

<code>docker build -t \${IMAGE_NAME}:local .</code>	build the image locally
<code>docker tag \${IMAGE_NAME}:local \${IMAGE_NAME}:\${IMAGE_TAG}</code>	tag with build-\${BUILD_NUMBER}
<code>docker push \${IMAGE_NAME}:\${IMAGE_TAG}</code>	push immutable tag to registry
<code>echo \$DOCKER_PASS docker login --password-stdin</code>	registry login without -p exposure
<code>terraform workspace select \${ENV_NAME} terraform workspace new \${ENV_NAME}</code>	isolate environment state
<code>terraform init</code>	download providers incl. FLOCI endpoints

Pipeline one-liners

<code>@Library('shared-library') _</code>	import the shared library, no auto globals
<code>env.BRANCH_NAME</code>	branch-conditional when in Multibranch
<code>\${{ needs.image.outputs.digest }}</code>	the one digest promoted everywhere
<code>credentialsId: 'github-token'</code>	PAT binding for private checkout
<code>tools/approved.txt</code>	presence flips the UAT-gated stage

NOTE Diagnostic order: failing check first — read it, reproduce locally, fix, re-verify. Never bypass a red gate.

Threads: [Controllers & Agents](#) · [Artifact Digest](#) · [Hermetic Builds](#)



Self-Quiz + Answer Key

Q1 Which file makes npm ci hermetic?

Q2 What blocks merging on a red main?

Q3 Which keyword orders test after build?

Q4 Tag or digest — what do you promote?

Q5 Rollback in one move?

Q6 Where must Jenkins builds run?

Q7 The webhook URL ends with what?

Q8 What runs before the human approval in the IaC pipeline?

▪ Answer key

1. package-lock.json.
2. Branch protection requiring the green build check.
3. needs: build.
4. The digest; :latest is a movable pointer.
5. Redeploy the previous known-good digest; no rebuild.
6. On agents; the controller keeps zero executors.
7. /github-webhook/ with the trailing slash.
8. Plan only; apply waits for the Approve input.

FINAL CHECK Eight for eight from memory, twice in a row — the labs are yours. Re-run any red lab until it is green.

Threads: [Controllers & Agents](#)

FINAL CHECK Sixty-four sheets strong — keep the [Command Cheatsheet](#) open beside every lab.